



в этом файле нет прямой речи, это инструкции для вас работайте в папке проекта в .env файле рас-
положите ваш api key из <https://huggingface.co/settings/tokens> HF_TOKEN=

расположите в project\scripts\https://github.com/tsenturion/rag/blob/main/scripts/download_hf_model.py
скачайте модель через питон файл, модель будет находиться в project\models\qwen

воспользуйтесь промптом в обычном чате

Создай полный учебный проект простого AI-агента на Python с использованием LangGraph.

Цель проекта: создать агента, который анализирует кодовую базу, находит места использования устаревшего метода и предлагает варианты замены.

Проект должен быть рассчитан на локальный запуск без использования облачных API.

В качестве языковой модели используй локальную модель Qwen2.5-1.5B-Instruct.

Модель находится в проекте по пути:

```
project/models/qwen
```

Где project – корневая директория проекта.

Создай структуру проекта:

```
project/
|
├─ models/
|   └─ qwen/
|
├─ agent/
|   ├── main.py
|   ├── graph.py
|   ├── nodes.py
|   ├── tools.py
|   └─ prompts.py
|
├─ test_project/
|   ├── old_code.py
|   └─ example.py
|
├─ requirements.txt
└─ README.md
```

Требования к реализации:

1. Используй LangGraph для создания графа агента.

2. Агент должен иметь несколько этапов работы:

- получение задачи пользователя;
- анализ файлов проекта;
- поиск устаревшего метода;
- формирование предложений по замене;
- создание итогового отчёта.

3. Реализуй простой цикл агента:

Агент получает цель.

Агент анализирует доступные файлы.

Агент принимает решение о следующих действиях.

Агент формирует результат.

Если информации недостаточно, агент должен запросить дополнительный анализ.

4. Создай инструменты агента:

Инструмент поиска файлов.

Инструмент чтения содержимого файла.

Инструмент поиска указанного метода.

Инструмент формирования отчёта.

5. Для демонстрации используй учебный пример.

Создай файл `test_project/old_code.py`:

В нём должен находиться устаревший метод:

```
old_function()
```

и новый рекомендуемый метод:

```
new_function()
```

Создай ещё несколько файлов, где старый метод используется несколько раз.

6. Агент должен выполнять задачу:

"Найди все места, где используется `old_function`, объясни проблему и предложи заменить его на `new_function`."

7. Используй локальную модель через `transformers`.

Не используй:

- OpenAI API;
- внешние серверы;
- платные сервисы;
- внешние ключи.

8. Код должен быть максимально простым и понятным для обучения.

Не создавай сложную промышленную архитектуру.

Каждый файл должен иметь комментарии на русском языке.

Добавь объяснение назначения каждого файла.

9. В `requirements.txt` добавь только необходимые библиотеки.

Используй:

- `langgraph`;
- `langchain`;
- `transformers`;
- `torch`.

10. Создай `README.md` с инструкцией:

- создание виртуального окружения;
- установка зависимостей;
- расположение модели;
- запуск агента;
- пример результата работы.

11. Код должен быть полностью готов к запуску.

Не используй псевдокод.

Не оставляй места вида:

"здесь добавьте код"

или

"реализуйте самостоятельно".

Все функции должны иметь рабочую реализацию.

После генерации сначала покажи структуру проекта, затем каждый файл полностью с кодом.

протестируйте полученный агент. Если получен отрицательный результат, то можете либо исправить ошибки также через интерфейс чата, либо через copilot/opencode

в любом случае покажите слушателям возможности самописного агента (если работает), copilot/opencode/hermes
